Experiment 'p_4_2_d_adjacent_duplicates_array_without_4th_ensures' Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_4_2_d_adjacent_duplicates_array_without_4th_ensures
Natural language statement: Write a method that takes a sequence of integers and returns all adjacent duplicates. For example, if the input is [1, 3, 3, 4, 5, 5, 6, 6, 6, 2], the method should return [3, 5, 6].
Method signature: p_4_2_d_adjacent_duplicates_array_without_4th_ensures(inputs: array<int>) returns (duplicates: array<int>)

Ensures

- `forall d :: d in duplicates[..] ==> d in inputs[..]`
- `forall i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates[..]`
- `forall d :: d in duplicates[..] ==> exists i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1] && inputs[i] == d`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns all adjacent duplicates. For
    ↪  example, if the input is [1, 3, 3, 4, 5, 5, 6, 6, 6, 2], the method should return
    ↪  [3, 5, 6].

The signature should be:

p_4_2_d_adjacent_duplicates_array_without_4th_ensures(inputs: array<int>) returns (
    ↪ duplicates: array<int>)

The method should respect the following contract:

ensures forall d :: d in duplicates[..] ==> d in inputs[..], ensures forall i :: 0 <= i <
    ↪  inputs.Length - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates[..],
    ↪ ensures forall d :: d in duplicates[..] ==> exists i :: 0 <= i < inputs.Length - 1
    ↪  && inputs[i] == inputs[i + 1] && inputs[i] == d

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn42CgTpAbtBpj6q5fKGSv2qktklu
Model: gpt-5.5-2026-04-23
Created at: 1780585952
Finish reason: stop
Usage: 546 tokens in, and 5654 tokens out

```
{
  var n := inputs.Length;
  var s: seq<int> := [];
  var i := 0;

  while i < n - 1
    invariant n == inputs.Length
    invariant 0 <= i <= n
    invariant forall d :: d in s ==> exists k :: 0 <= k < i && k < n - 1 && inputs[k] ==
        ↪ inputs[k + 1] && inputs[k] == d
    invariant forall k :: 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] ==> inputs
        ↪ [k] in s
  {
    var prev := s;
```

```
  if inputs[i] == inputs[i + 1] {
    s := s + [inputs[i]];
  }

  forall d | d in s
    ensures exists k :: 0 <= k < i + 1 && k < n - 1 && inputs[k] == inputs[k + 1] &&
        ↪ inputs[k] == d
  {
    if d in prev {
      var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
      assert 0 <= k < i + 1;
    } else {
      if inputs[i] == inputs[i + 1] {
        assert s == prev + [inputs[i]];
        assert d == inputs[i];
        assert 0 <= i < i + 1;
        assert i < n - 1;
      } else {
        assert s == prev;
        assert d in prev;
      }
    }
  }

  forall k | 0 <= k < i + 1 && k < n - 1 && inputs[k] == inputs[k + 1]
    ensures inputs[k] in s
  {
    if k < i {
      assert inputs[k] in prev;
      if inputs[i] == inputs[i + 1] {
        assert s == prev + [inputs[i]];
        assert inputs[k] in s;
      } else {
        assert s == prev;
        assert inputs[k] in s;
      }
    } else {
      assert k == i;
      assert inputs[i] == inputs[i + 1];
      assert s == prev + [inputs[i]];
      assert inputs[k] == inputs[i];
      assert inputs[k] in s;
    }
  }

  i := i + 1;
}

duplicates := new int[|s|](j => s[j]);
assert duplicates.Length == |s|;
assert forall j :: 0 <= j < |s| ==> duplicates[j] == s[j];
```

```
  assert duplicates[..] == s;

  forall d | d in duplicates[..]
    ensures d in inputs[..]
  {
    assert d in s;
    var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
    assert n == inputs.Length;
    assert 0 <= k < inputs.Length;
    assert inputs[..][k] == inputs[k];
    assert d in inputs[..];
  }

  forall k | 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1]
    ensures inputs[k] in duplicates[..]
  {
    assert n == inputs.Length;
    assert !(i < n - 1);
    assert i >= n - 1;
    assert k < n - 1;
    assert k < i;
    assert inputs[k] in s;
    assert inputs[k] in duplicates[..];
  }

  forall d | d in duplicates[..]
    ensures exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs
        ↪ [k] == d
  {
    assert d in s;
    var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
    assert n == inputs.Length;
    assert 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(65,35):
    ↪ Error: index out of range
   |
65 | duplicates := new int[|s|](j => s[j]);
   |                            ^


Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
```

```
    ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn44GCIgnNDRA7XdQMqBZKIqGYy4U
Model: gpt-5.5-2026-04-23
Created at: 1780586080
Finish reason: stop
Usage: 1574 tokens in, and 4055 tokens out

```
{
  var n := inputs.Length;
  var s: seq<int> := [];
  var i := 0;

  while i < n - 1
    invariant n == inputs.Length
    invariant 0 <= i <= n
    invariant forall d :: d in s ==> exists k :: 0 <= k < i && k < n - 1 && inputs[k] ==
        ↪ inputs[k + 1] && inputs[k] == d
    invariant forall k :: 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] ==> inputs
        ↪ [k] in s
  {
    var prev := s;

    if inputs[i] == inputs[i + 1] {
      s := s + [inputs[i]];
    }

    forall d | d in s
      ensures exists k :: 0 <= k < i + 1 && k < n - 1 && inputs[k] == inputs[k + 1] &&
          ↪ inputs[k] == d
    {
      if d in prev {
        var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
        assert 0 <= k < i + 1;
      } else {
        if inputs[i] == inputs[i + 1] {
          assert s == prev + [inputs[i]];
          assert d == inputs[i];
          assert 0 <= i < i + 1;
          assert i < n - 1;
        } else {
          assert s == prev;
          assert d in prev;
        }
      }
    }

    forall k | 0 <= k < i + 1 && k < n - 1 && inputs[k] == inputs[k + 1]
```

```
    ensures inputs[k] in s
  {
    if k < i {
      assert inputs[k] in prev;
      if inputs[i] == inputs[i + 1] {
        assert s == prev + [inputs[i]];
        assert inputs[k] in s;
      } else {
        assert s == prev;
        assert inputs[k] in s;
      }
    } else {
      assert k == i;
      assert inputs[i] == inputs[i + 1];
      assert s == prev + [inputs[i]];
      assert inputs[k] == inputs[i];
      assert inputs[k] in s;
    }
  }

  i := i + 1;
}

assert i >= n - 1;

duplicates := new int[|s|];
assert duplicates.Length == |s|;
assert duplicates != inputs;

var j := 0;
while j < |s|
  invariant n == inputs.Length
  invariant 0 <= i <= n
  invariant i >= n - 1
  invariant duplicates != inputs
  invariant duplicates.Length == |s|
  invariant 0 <= j <= |s|
  invariant duplicates[..j] == s[..j]
  invariant forall d :: d in s ==> exists k :: 0 <= k < i && k < n - 1 && inputs[k] ==
      ↪ inputs[k + 1] && inputs[k] == d
  invariant forall k :: 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] ==> inputs
      ↪ [k] in s
{
  duplicates[j] := s[j];
  assert duplicates[j] == s[j];
  assert duplicates[..j] == s[..j];
  assert duplicates[..j + 1] == s[..j + 1];
  j := j + 1;
}

assert j == |s|;
```

```
  assert duplicates[..] == duplicates[..j];
  assert s[..j] == s;
  assert duplicates[..] == s;

  forall d | d in duplicates[..]
    ensures d in inputs[..]
  {
    assert d in s;
    var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
    assert n == inputs.Length;
    assert 0 <= k < inputs.Length;
    assert inputs[..][k] == inputs[k];
    assert d in inputs[..];
  }

  forall k | 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1]
    ensures inputs[k] in duplicates[..]
  {
    assert n == inputs.Length;
    assert !(i < n - 1);
    assert i >= n - 1;
    assert k < n - 1;
    assert k < i;
    assert inputs[k] in s;
    assert inputs[k] in duplicates[..];
  }

  forall d | d in duplicates[..]
    ensures exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs
        ↪ [k] == d
  {
    assert d in s;
    var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
    assert n == inputs.Length;
    assert 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
  }
}
```

Final Program

```
method p_4_2_d_adjacent_duplicates_array_without_4th_ensures(inputs: array<int>) returns
    ↪ (duplicates: array<int>)
      ensures forall d :: d in duplicates[..] ==> d in inputs[..]
      ensures forall i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1] ==>
          ↪ inputs[i] in duplicates[..]
      ensures forall d :: d in duplicates[..] ==> exists i :: 0 <= i < inputs.Length - 1
          ↪  && inputs[i] == inputs[i + 1] && inputs[i] == d
{
  var n := inputs.Length;
  var s: seq<int> := [];
```

```
var i := 0;

while i < n - 1
  invariant n == inputs.Length
  invariant 0 <= i <= n
  invariant forall d :: d in s ==> exists k :: 0 <= k < i && k < n - 1 && inputs[k] ==
      ↪ inputs[k + 1] && inputs[k] == d
  invariant forall k :: 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] ==> inputs
      ↪ [k] in s
{
  var prev := s;

  if inputs[i] == inputs[i + 1] {
    s := s + [inputs[i]];
  }

  forall d | d in s
    ensures exists k :: 0 <= k < i + 1 && k < n - 1 && inputs[k] == inputs[k + 1] &&
        ↪ inputs[k] == d
  {
    if d in prev {
      var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
      assert 0 <= k < i + 1;
    } else {
      if inputs[i] == inputs[i + 1] {
        assert s == prev + [inputs[i]];
        assert d == inputs[i];
        assert 0 <= i < i + 1;
        assert i < n - 1;
      } else {
        assert s == prev;
        assert d in prev;
      }
    }
  }

  forall k | 0 <= k < i + 1 && k < n - 1 && inputs[k] == inputs[k + 1]
    ensures inputs[k] in s
  {
    if k < i {
      assert inputs[k] in prev;
      if inputs[i] == inputs[i + 1] {
        assert s == prev + [inputs[i]];
        assert inputs[k] in s;
      } else {
        assert s == prev;
        assert inputs[k] in s;
      }
    } else {
      assert k == i;
      assert inputs[i] == inputs[i + 1];
```

```
      assert s == prev + [inputs[i]];
      assert inputs[k] == inputs[i];
      assert inputs[k] in s;
    }
  }

  i := i + 1;
}

assert i >= n - 1;

duplicates := new int[|s|];
assert duplicates.Length == |s|;
assert duplicates != inputs;

var j := 0;
while j < |s|
  invariant n == inputs.Length
  invariant 0 <= i <= n
  invariant i >= n - 1
  invariant duplicates != inputs
  invariant duplicates.Length == |s|
  invariant 0 <= j <= |s|
  invariant duplicates[..j] == s[..j]
  invariant forall d :: d in s ==> exists k :: 0 <= k < i && k < n - 1 && inputs[k] ==
      ↪ inputs[k + 1] && inputs[k] == d
  invariant forall k :: 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] ==> inputs
      ↪ [k] in s
{
  duplicates[j] := s[j];
  assert duplicates[j] == s[j];
  assert duplicates[..j] == s[..j];
  assert duplicates[..j + 1] == s[..j + 1];
  j := j + 1;
}

assert j == |s|;
assert duplicates[..] == duplicates[..j];
assert s[..j] == s;
assert duplicates[..] == s;

forall d | d in duplicates[..]
  ensures d in inputs[..]
{
  assert d in s;
  var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
  assert n == inputs.Length;
  assert 0 <= k < inputs.Length;
  assert inputs[..][k] == inputs[k];
  assert d in inputs[..];
}
```

```
  forall k | 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1]
    ensures inputs[k] in duplicates[..]
  {
    assert n == inputs.Length;
    assert !(i < n - 1);
    assert i >= n - 1;
    assert k < n - 1;
    assert k < i;
    assert inputs[k] in s;
    assert inputs[k] in duplicates[..];
  }

  forall d | d in duplicates[..]
    ensures exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs
        ↪ [k] == d
  {
    assert d in s;
    var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
    assert n == inputs.Length;
    assert 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
  }
}
```

Total Token Usage

Input tokens: 2120
Output tokens: 9709
Reasoning tokens: 7386
Sum of 'total tokens': 11829

Experiment Timings

Overall Experiment started at 1780585951459, ended at 1780586194095, lasting 242636ms (242.64 seconds)
Iteration #1 started at 1780585951475, ended at 1780586079268, lasting 127793ms (127.79 seconds)
Iteration #2 started at 1780586079340, ended at 1780586194094, lasting 114754ms (114.75 seconds)